

## 1 3.E - Protocoles de Transport (TCP, UDP) et protocoles de contrôle (ICMP, ARP, DHCP)

Les **protocoles de transport** servent à véhiculer les informations qui proviennent d'une application Internet comme le Web, le transfert de fichiers, le courrier électronique et d'autres applications qui vous ont été présentées. Les **protocoles de transport** d'Internet sont **TCP** et **UDP**.

Les **protocoles de contrôle** de l'Internet ne sont pas directement utilisés par une application mais sont utiles au fonctionnement d'Internet. Le protocole **ICMP** permet de tester l'échange de paquets IP entre une adresse IP source et une adresse IP destination. Il est utilisé par les utilitaires *ping* et *tracert*. Le protocole **ARP** est essentiel au fonctionnement d'Internet car il permet de faire le lien entre les adresses IP qui sont utilisées pour trouver le chemin vers une destination sur Internet et les adresses MAC qui permettent à un signal d'aller d'une carte réseau à une autre carte réseau sur une liaison. Enfin, le protocole **DHCP** permet de simplifier la configuration des équipements en leur attribuant leur configuration réseau (adresse IP, masque, passerelle, etc) sur simple demande.

Nous présentons d'abord les protocoles de transport puis les protocoles de contrôle.

### 1.1 Le protocole UDP

Comme cela a été vu dans une autre section, le protocole **UDP** apporte principalement la rapidité et la souplesse d'envoi. C'est-à-dire que l'application qui va utiliser UDP ne sera pas limitée en termes d'émission et en termes de débit. Elle pourra envoyer beaucoup de données rapidement. Mais la contrepartie est qu'elle n'aura pas de garantie sur le fait que les données arrivent correctement. A contrario, TCP apporte la fiabilité.

Le protocole **UDP (RFC 768)** est donc très simple. Il prend les données de l'application et ajoute un en-tête devant qui contient les numéros de port source et destination ainsi qu'un checksum. Les numéros de port permettent d'identifier l'application sur l'émetteur et sur le récepteur. La somme de contrôle permet à UDP de faire le contrôle des erreurs et donc vérifier quand un paquet arrive qu'il n'a pas été altéré pendant son transport. Dans le cas où une erreur est détectée, le paquet est détruit.

#### 1.1.1 La rapidité de UDP

UDP est en mode **non connecté**, c'est-à-dire que lorsque l'application veut envoyer un paquet, elle ne demande pas au destinataire, donc à l'application réceptrice, si elle est en mesure de recevoir le message. Avec UDP, l'émetteur envoie le paquet sans demander l'autorisation au récepteur.

Pourquoi avons-nous besoin d'un protocole comme UDP ? Tout simplement pour avoir des échanges applicatifs qui puissent, si nécessaire, être rapides, sans créer une connexion, sans vérifier que tout ce qui arrive côté récepteur est exactement ce qu'il y avait côté émetteur, ce qui nécessite des algorithmes complexes qui prennent du temps. Donc UDP apporte la rapidité et une meilleure bande passante, c'est-à-dire un meilleur débit, mais aucune garantie de bon acheminement pour l'application qui l'utilise.

#### 1.1.2 L'entête UDP

L'entête UDP fait 8 octets.

Il contient le **port source** et le **port de destination** pour identifier l'application sur l'émetteur et sur le récepteur, la **longueur du segment** (soit l'entête UDP de 8 octets + la taille des données) et un **checksum** qui permet de faire la détection d'erreur. Un numéro de port est codé sur deux octets donc sa valeur varie entre 0 et 65535. Les ports des serveurs sont fixés par la norme et inférieurs à 1024 alors que les ports des clients sont attribués dynamiquement par le système d'exploitation de la machine cliente et compris entre 1024 et 65535.

#### 1.1.3 UDP en mode non connecté

UDP fonctionne en mode non connecté.

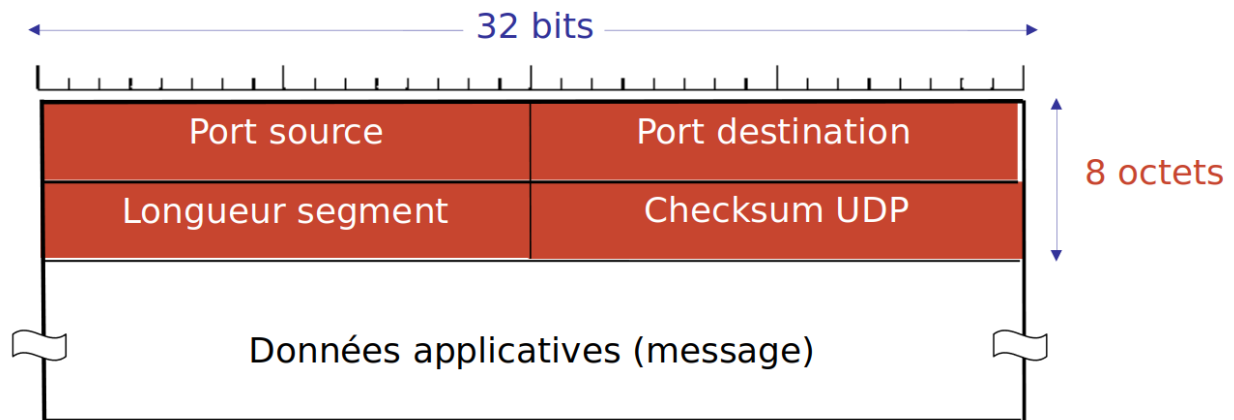


Figure 1: Entête UDP

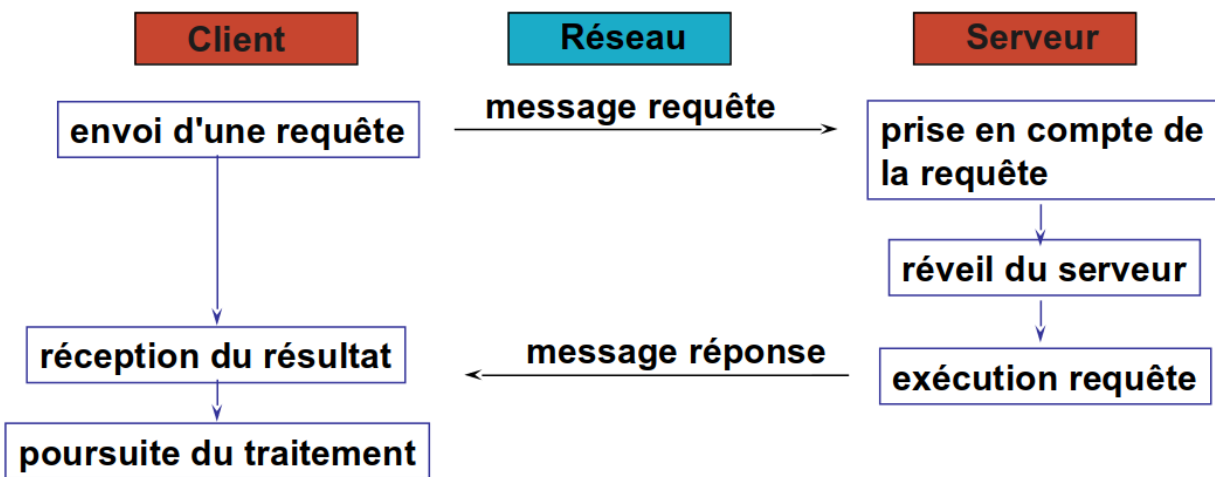


Figure 2: Schéma UDP

Cela signifie que, quand une application veut envoyer un message, elle le fait immédiatement, sans attendre de savoir si le récepteur est prêt à recevoir la requête. Le client envoie sa requête en espérant qu'elle arrive à destination. Si elle arrive, le serveur se réveille, traite la requête et répond éventuellement. Il peut aussi ne pas répondre. C'est très simple et équivalent à l'envoi d'un courrier électronique ou d'une lettre par la poste. Quand on le fait, on ne demande pas au destinataire s'il est d'accord pour recevoir le message ou la lettre qu'on lui envoie.

#### 1.1.4 Les applications qui utilisent UDP

Les applications qui utilisent UDP sont principalement toutes les applications multimédias qui transportent de la vidéo, du son, de la musique, etc. donc par exemple la *streaming*, la visioconférence ou la voix sur IP, c'est-à-dire la téléphonie sur Internet. Pourquoi ces applications-là ? Parce qu'elles sont tolérantes aux pertes et aux erreurs. Ainsi, on peut se permettre, dans une image ou dans une bande son, d'avoir des paquets qui sont perdus ou des erreurs qui apparaissent, sachant que l'œil humain ou l'oreille humaine ne s'apercevront pas de ces erreurs. Cela permet aussi à ces applications de disposer d'un meilleur débit. L'important pour les applications multimédia, c'est que les données arrivent à la bonne vitesse : il y a un débit à respecter pour que la communication vocale soit audible ou la vidéo fluide.

Un autre exemple d'application sur Internet qui n'utilise pas TCP pour des raisons de rapidité est le DNS. Il permet de faire la correspondance entre les noms de machines et les adresses IP sur Internet. C'est un protocole essentiel qui est utilisé par toutes les applications. Pourquoi utilise-t-il UDP ? Il transporte des messages qui ont une petite taille et qui nécessitent une réponse rapide. Ainsi, on va s'affranchir de la connexion TCP pour envoyer les requêtes et les réponses DNS en utilisant UDP. Cela n'empêche pas que ce protocole a besoin de fiabilité en principe. Ici, la fiabilité est reportée au niveau de l'application elle-même. C'est-à-dire que si une application envoie une requête DNS et que le checksum UDP détecte une erreur, alors l'application va refaire sa requête, donc faire la retransmission elle-même, alors que sinon c'est géré par TCP directement.

Terminons en citant une dernière application qui utilise UDP toujours pour les mêmes raisons : NFS (*Network File System*) qui permet de partager des fichiers sur un réseau Linux.

## 1.2 Le protocole TCP

Nous allons maintenant décrire le protocole TCP qui réalise la fiabilité en plus de faire le lien avec les applications. Comme UDP, TCP utilise les numéros de ports qui permettent d'identifier l'application émettrice et réceptrice. Il contient dans son entête le port source et le port destination de l'application.

TCP transporte un **flux d'octets** qui provient de l'application. Cela signifie que la notion de message disparaît au niveau de TCP : TCP prend un ensemble d'octets issu d'un message applicatif et les envoie de manière fiable. Nous allons revoir ce qu'est la fiabilité, comment mettre en place des mécanismes pour la mettre en œuvre et le fonctionnement de TCP dans toute sa complexité.

L'application lit ou écrit des octets dans une **socket** dédiée à cette application. Une *socket* correspond à des zones mémoire pour envoyer et recevoir des données. Ensuite, TCP s'occupe de faire que tout arrive dans l'ordre, sans erreur et sans perte. Pour cela, il effectue des retransmissions si nécessaire, au bout d'un certain temps, quand les accusés de réception n'arrivent pas.

TCP réalise également le **contrôle de flux**. Il permet de n'envoyer un message qu'à la condition que le récepteur soit en capacité de recevoir ce message. Par exemple, si un serveur est bloqué pour une raison ou une autre et ne peut plus recevoir, le contrôle de flux va demander au client d'arrêter d'envoyer des données pour éviter qu'elles ne soient perdues.

TCP met aussi en œuvre le **contrôle de congestion**. Il permet à TCP de ne pas envoyer des données dans le réseau s'il y a une congestion, c'est-à-dire un point d'encombrement dans le réseau. Le principe est le suivant : si un ou plusieurs messages ne sont pas acquittés, TCP suppose qu'ils ont été perdus à cause d'une congestion ; de ce fait, il ralentit ses émissions.

TCP est composé de plusieurs RFC ([RFC 793](#), [RFC 1122](#), [RFC 1323](#), [RFC 2018](#) et [RFC 2581](#)). Il y a plusieurs documents qui normalisent ce protocole avec des algorithmes complexes et de nombreuses évolutions au fil des années.

### 1.2.1 La fiabilité de TCP

Le **rôle de TCP** est de prendre des données d'une application et de faire en sorte qu'elles arrivent de manière fiable, c'est-à-dire que tout ce qui arrive sur le récepteur soit exactement ce qui a été envoyé par l'émetteur. Pour ce faire, TCP est en mode connecté : le client doit faire une demande de connexion au serveur avant d'envoyer ses données. Le serveur peut l'accepter ou la refuser. S'il l'accepte, les données vont être transportées de manière fiable pendant toute la durée de la connexion. La demande de fermeture de la connexion peut être à l'initiative soit du client soit du serveur.

La fiabilité consiste à vérifier qu'il n'y a pas de perte, pas d'erreur et, en cas d'erreur, à retransmettre les octets erronés. Il faut également vérifier que les données arrivent dans le bon ordre. En effet, étant donné que des retransmissions peuvent être mises en œuvre, les messages peuvent être dupliqués et/ou arriver dans le désordre. Il faut également pouvoir vérifier que les paquets n'arrivent pas en double. En effet, il peut y avoir des doubles, car des retransmissions peuvent entraîner l'envoi du même paquet plusieurs fois et donc créer des doubles. Il faut donc le gérer.

Comment réaliser la fiabilité ? Des retransmissions sont nécessaires pour retransmettre les paquets erronés ou perdus. Un **checksum** est nécessaire pour vérifier qu'il n'y a pas d'erreur dans un paquet reçu. Le checksum est un calcul effectué sur les données avant l'émission. Il permet d'obtenir une valeur : c'est la somme de contrôle. Le récepteur refait le même calcul sur les données reçues et, en fonction de cela, compare le checksum avant envoi et le checksum sur les données reçues. Si les deux valeurs sont différentes, cela signifie qu'il y a une erreur dans le paquet.

Pour pouvoir effectuer des retransmissions en cas d'erreur ou de perte, il est nécessaire de stocker tous les messages envoyés jusqu'à ce qu'un **acquittement** soit reçu. C'est pourquoi un nouveau mécanisme, l'acquittement (noté ACK pour *Aknowledge*), est introduit. Il permet de dire quand un message a été bien reçu et donc de le libérer côté émetteur pour arrêter de le stocker en vue d'une éventuelle retransmission.

L'émetteur doit donc stocker tous les messages envoyés non acquittés. Un timeout est également nécessaire. Le **timeout** est une temporisation qui, pour chaque message envoyé, déclenche un chronomètre et, lorsque ce dernier arrive à zéro, si aucun acquittement n'a été reçu, le message est retransmis. Si un paquet est erroné ou perdu, il n'est pas acquitté et retransmis au bout du *timeout*.

Pour pouvoir gérer l'ordre des messages envoyés et éviter les duplications, il est nécessaire de **numéroter les messages envoyés et les acquittements**. TCP fait du **piggybacking** : chaque message contient dans son entête le numéro du message envoyé et le numéro du message acquitté. Voilà pour les mécanismes qui permettent de garantir la fiabilité.

### 1.2.2 TCP en mode connecté

TCP est en mode connecté : le client, avant d'envoyer des données au serveur, doit faire une demande d'ouverture de connexion. Les ports des serveurs sont connus des clients car ils sont fixés par la norme : [Numéros de port réservés](#).

Si le serveur est d'accord, il crée un contexte pour cette connexion. Une fois la connexion ouverte, le client et le serveur peuvent s'échanger des données qui seront transmises de manière fiable. Lorsque l'un ou l'autre souhaite interrompre la connexion, c'est-à-dire y mettre fin, il fait une demande de déconnexion qui, selon le contexte, peut être acceptée ou refusée.

Le mode connecté est donc comparable à un appel téléphonique. Si vous appelez quelqu'un, ça sonne et le destinataire n'est pas obligé de répondre, mais s'il le fait, la connexion est établie et les deux personnes peuvent se parler. Il faut noter que TCP est le seul protocole d'Internet en mode connecté que nous étudions dans le cadre de ce cours. Tous les autres protocoles, que ce soit UDP, ICMP, IP, ARP ou les protocoles

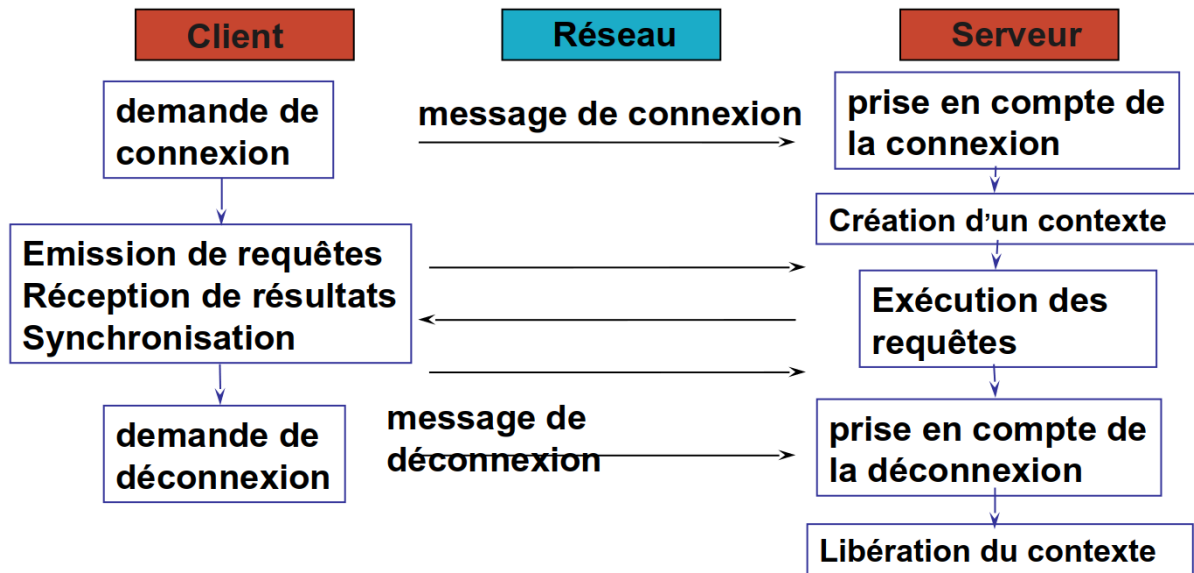


Figure 3: Dialog TCP

applicatifs, ne réalisent pas de connexion. On parle souvent d'une connexion web, ce qui est une erreur : le protocole HTTP n'est pas en mode connecté.

### 1.2.3 La connexion TCP

Quand une application a besoin de communiquer sur Internet, c'est le client qui contacte le serveur. Le client et le serveur utilisent chacun une **socket** pour envoyer/recevoir des données vers/depuis le réseau. Une socket est un fichier virtuel dans lequel le processus client ou serveur lit ou écrit des données selon qu'il veut recevoir ou envoyer des données. TCP se trouve dans le système d'exploitation. Il gère la demande de connexion, la fiabilité des données transmises et la fermeture de la connexion.

Une **connexion TCP** est identifiée par le quadruplet : adresse IP source, port source, adresse IP destination, port destination.

Dans l'exemple ci-dessous, le client est en haut car il utilise un port supérieur à 1024, ici le port 5004. En bas se trouve le serveur qui utilise le port 80. Il s'agit donc d'une connexion entre un navigateur web en haut et un serveur Web en bas. L'application (le navigateur web d'un côté, le serveur Web de l'autre) est un programme qui va à travers son numéro de port lire ou écrire des données dans la socket. Concrètement, une **socket** est une zone d'émission (le *TCP send buffer*) et une zone de réception (le *TCP receive buffer*) de chaque côté. Quand l'application écrit dans la socket, ça vient déposer des données dans le *send buffer*. Ensuite, TCP va prendre en charge ces données pour les envoyer de manière fiable dans le *receive buffer* de l'autre côté.

Le contrôle de flux permet d'éviter que TCP envoie des données dans un *buffer* de réception déjà saturé. Il gère la capacité restante du *buffer* de réception et permet d'éviter d'envoyer des données qui pourraient être perdues s'il n'y a pas assez de place.

### 1.2.4 Ouverture d'une connexion TCP

La figure ci-dessous représente le processus d'ouverture de connexion TCP. Dans l'en-tête TCP qui sera présentée un peu plus loin figure des drapeaux ou *flags* en anglais qui permettent de "typer" certains messages TCP c'est-à-dire de leur donner une signification particulière. Chaque drapeau est un bit qui vaut zéro ou un. Pour une ouverture de connexion, les drapeaux **SYN** et **ACK** sont utilisés.

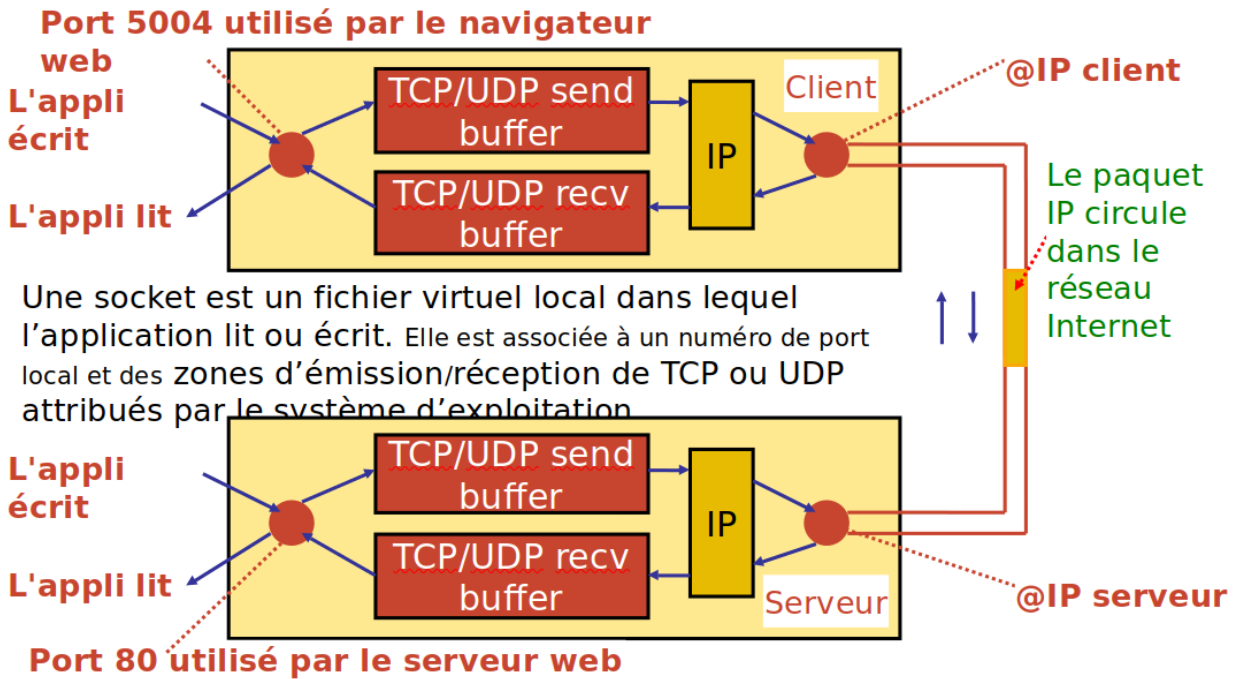


Figure 4: Buffers de TCP

L'ouverture se passe en 3 messages : SYN + SYN/ACK + ACK :

- Le client fait la demande d'ouverture avec SYN=1 (et ACK=0).
- S'il est d'accord, le serveur répond OK avec SYN=1 et ACK=1.
- Enfin, le client confirme l'ouverture de la connexion avec ACK=1.

Après ces 3 messages, il est possible de commencer à transmettre des données.

**SEQ** est le numéro de séquence. Il permet de numéroté tous les octets envoyés dans cette connexion. Ce numéro est initialisé aléatoirement à l'ouverture de la connexion.

### 1.2.5 Fermeture d'une connexion TCP

La fermeture de connexion suit un peu le même principe :

- Le client envoie un message TCP en positionnant le flag **FIN** à 1. C'est pour initier la demande de fermeture de connexion.
- Le serveur répond avec un message **ACK** pour dire "OK, j'ai bien reçu ta demande de fermeture".
- Sauf qu'il avait (potentiellement) encore des données à envoyer qui n'étaient pas encore reçues. Donc, avant la fermeture de la connexion, il les envoie.
- Le client acquitte les données qu'il vient de recevoir du serveur.
- Après cela, le serveur confirme la fermeture de connexion avec un FIN=1.
- Le dernier message confirme la fermeture par le client : ACK=1 et FIN=1.

Dans cet exemple, c'est le client qui demande la fermeture de la connexion. Le serveur peut lui aussi, de la même manière, initier la fermeture de la connexion.

Il est important de rappeler que TCP est un protocole **de bout en bout** : il existe uniquement aux extrémités, c'est-à-dire sur le client et le serveur. Les équipements intermédiaires (les routeurs, les commutateurs, les répéteurs, etc.) n'ont aucune visibilité de la connexion TCP. De plus, il faut s'imaginer que le routage des paquets peut éventuellement changer pendant la durée de vie de la connexion. Cela signifie que

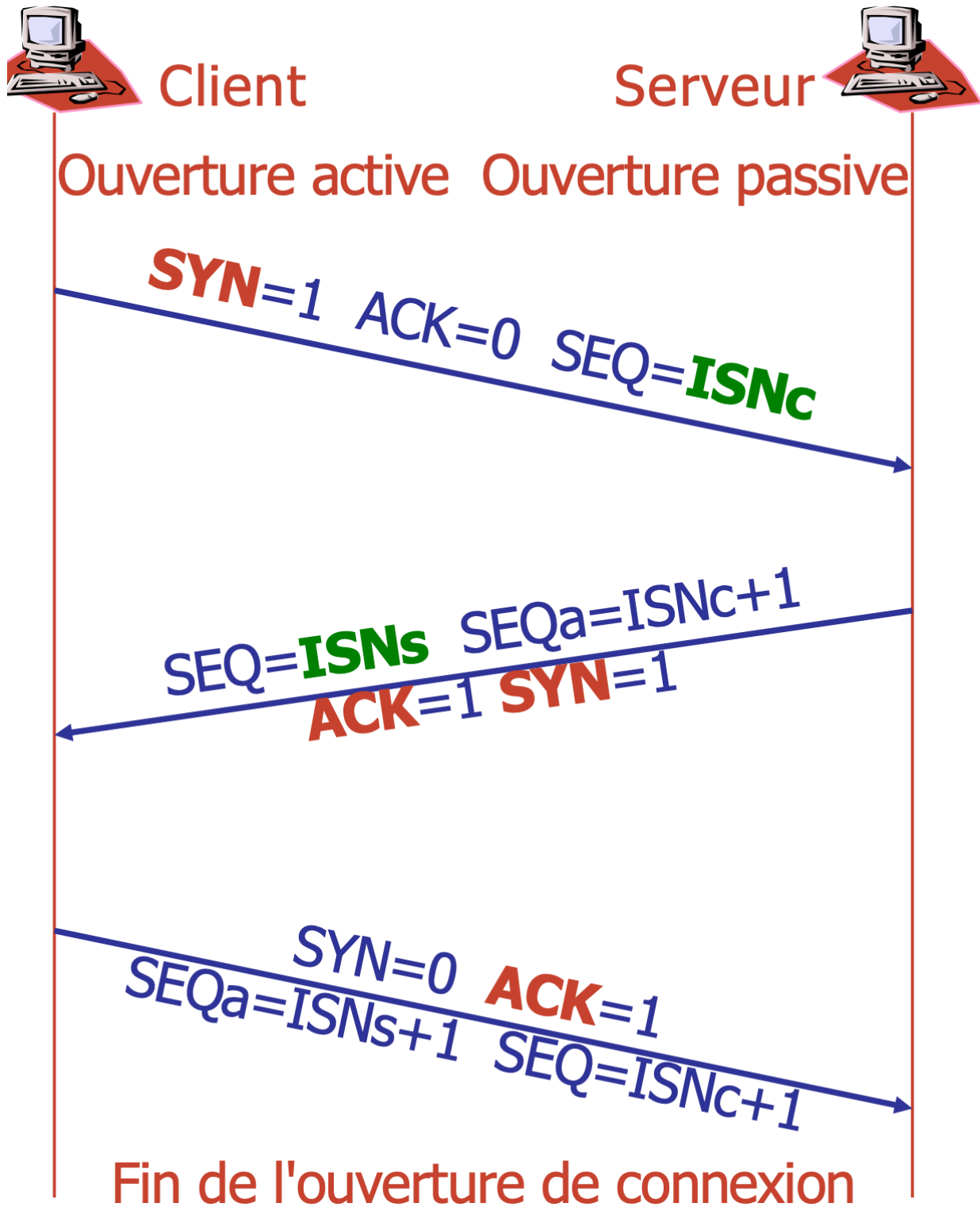


Figure 5: Ouverture d'une connexion TCP

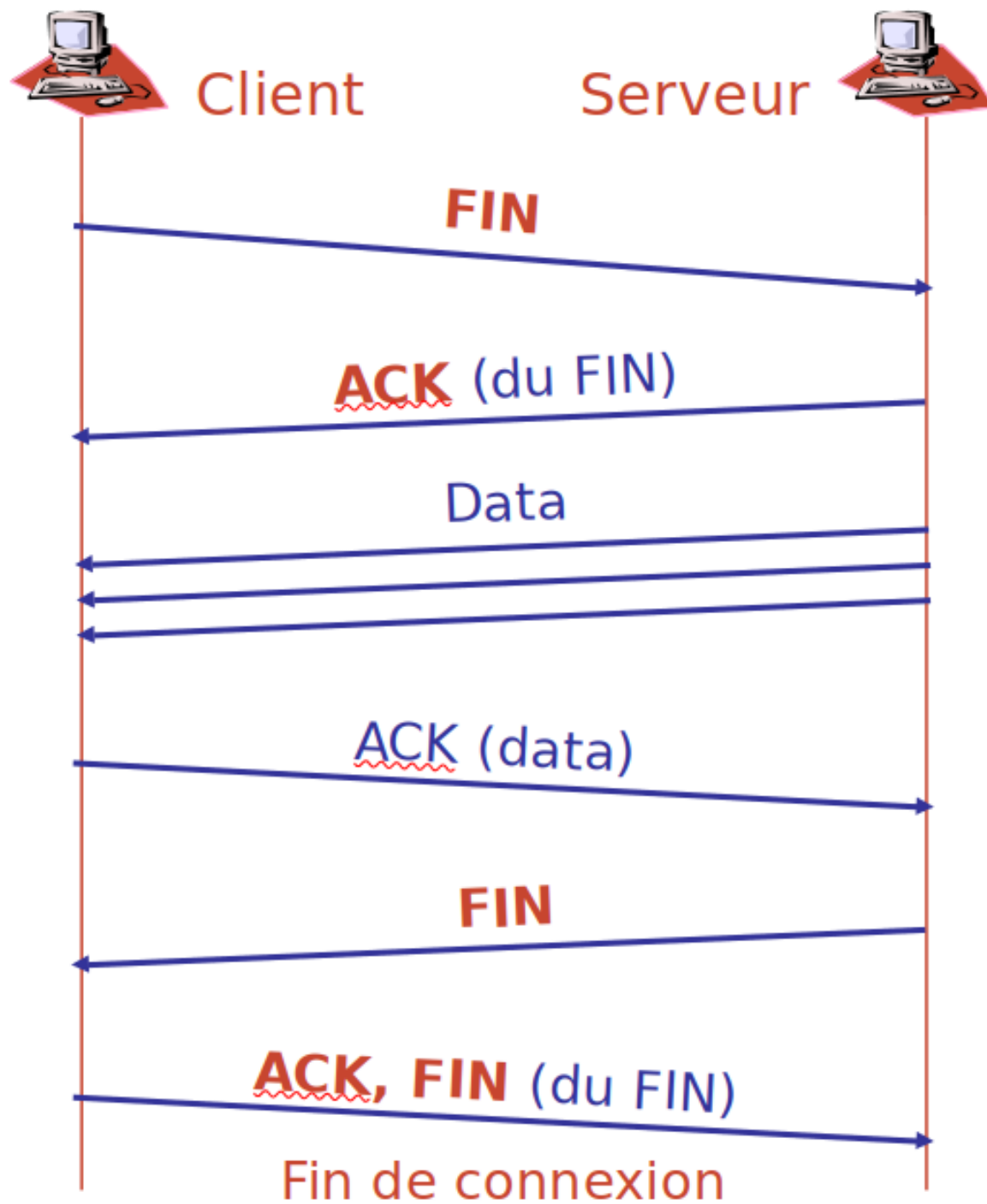


Figure 6: Fermeture d'une connexion TCP

tous les paquets qui vont transiter dans une même connexion ne vont pas forcément suivre le même chemin dans le réseau.

### 1.2.6 Les applications qui utilisent TCP

Les applications qui utilisent TCP sont toutes celles qui ont besoin de fiabilité, c'est-à-dire presque toutes les applications à l'exception des applications multimédia. Par exemple, si vous demandez une page web ou envoyez un courrier électronique, et qu'un seul bit change de valeur pendant le transfert d'un texte, l'utilisateur le verra dans la page web ou dans son courrier électronique, car le code ASCII d'une lettre aura été modifié. Par exemple, la lettre A pourrait devenir la lettre I.

Voici les applications classiques d'Internet avec leurs protocoles applicatifs qui ont besoin de la fiabilité et qui utilisent TCP :

- Le web (HTTP/HTTPS)
- La connexion à distance (Telnet, SSH, X)
- Le courrier électronique (SMTP, POP, IMAP, Webmail)
- Le transfert de fichiers (FTP)
- L'annuaire fédérateur (LDAP)

### 1.2.7 L'entête TCP

Nous allons maintenant examiner le contenu de l'entête TCP. Il contient ce qui permet de faire tout ce qui a été vu précédemment c'est-à-dire la fiabilité, le contrôle de flux, le contrôle de congestion, les ouvertures et fermetures de connexion, etc.

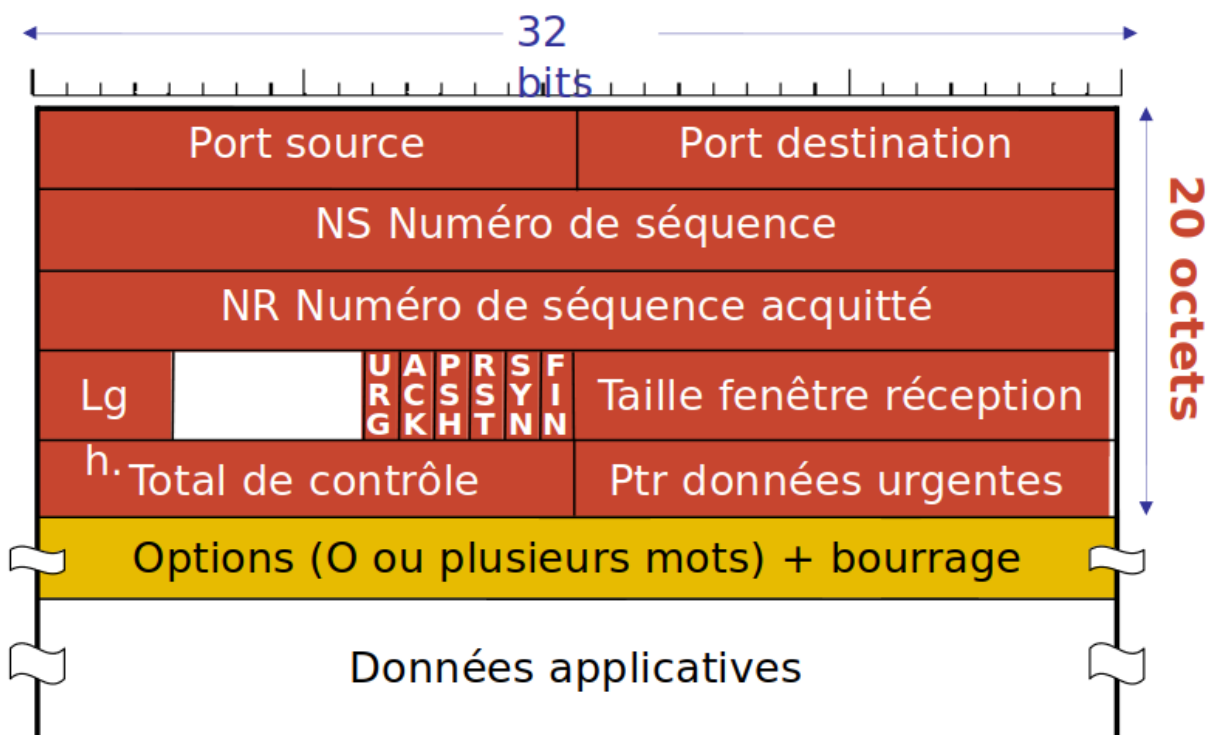


Figure 7: Entête TCP

Comme pour IP, nous avons un entête de 5 lignes, soit 20 octets.

- Comme pour UDP, l'entête commence par les **numéros de ports** du client et du serveur qui permettent d'identifier l'application concernée par le message. Le **port source/destination** est lié à une *socket* et donc une zone mémoire pour envoyer/recevoir des données.
- Les numéros **NS** et **NR** qui permettent de compter les octets envoyés et reçus, et donc de s'assurer que tout a bien été envoyé et reçu correctement.
- Comme pour IP, **Lg** contient la longueur de l'entête (**Lg**) c'est-à-dire le nombre de lignes de l'entête.
- Ensuite, six bits qui ne sont pas utilisés ; ils sont réservés pour une utilisation future.
- Viennent alors les **drapeaux** qui permet de typer certains messages. Le bit **ACK** s'il est à 1 indique qu'il s'agit d'un acquittement ; **SYN** permet d'initier une ouverture de connexion, **FIN** une fermeture de connexion ; **RST** est utilisé pour redémarrer la connexion, la réinitialiser, **URG** pour transporter des données urgentes c'est-à-dire qui ne sont pas soumises au contrôle de flux et au contrôle de congestion ; enfin, **PSH** (*push*) fait partir les données tout de suite, sans attendre : par défaut, pour gagner en efficacité, TCP attend d'avoir suffisamment d'octets dans la zone d'émission avant d'envoyer les données mais certaines applications ont besoin que chaque octet déposé parte aussitôt auquel cas PSH=1.
- La **taille de la fenêtre de réception** permet de mettre en oeuvre le contrôle de flux : elle indique le nombre d'octets qui peuvent être reçus. L'idée est que l'émetteur ne va pas envoyer des données s'il sait qu'elles ne peuvent pas être stockées en réception. Et donc, dans ce cas là, TCP va ralentir ses émissions et s'adapter à la taille de la fenêtre de réception, qui est transmise dans l'entête de tous les messages TCP.
- Le **total de contrôle** est le *checksum*, la somme de contrôle. Il permet de faire la détection d'erreur.
- Le pointeur sur les **données urgentes** est utilisé quand le bit URG vaut 1. Il permet d'envoyer des données sans passer par le contrôle de flux et le contrôle de congestion de TCP. Donc, c'est pour des situations particulières au niveau de certaines applications qui pourraient en avoir besoin. Ce champ stocke sur deux octets un pointeur qui va référencer une zone en mémoire dédiée aux données urgentes. Cela sert à faire en sorte que, même si le tampon de réception est plein, l'application peut tout de même envoyer une donnée urgente, typiquement, pour interrompre l'application ou lui signaler un problème.

### 1.2.8 Principe des acquittements

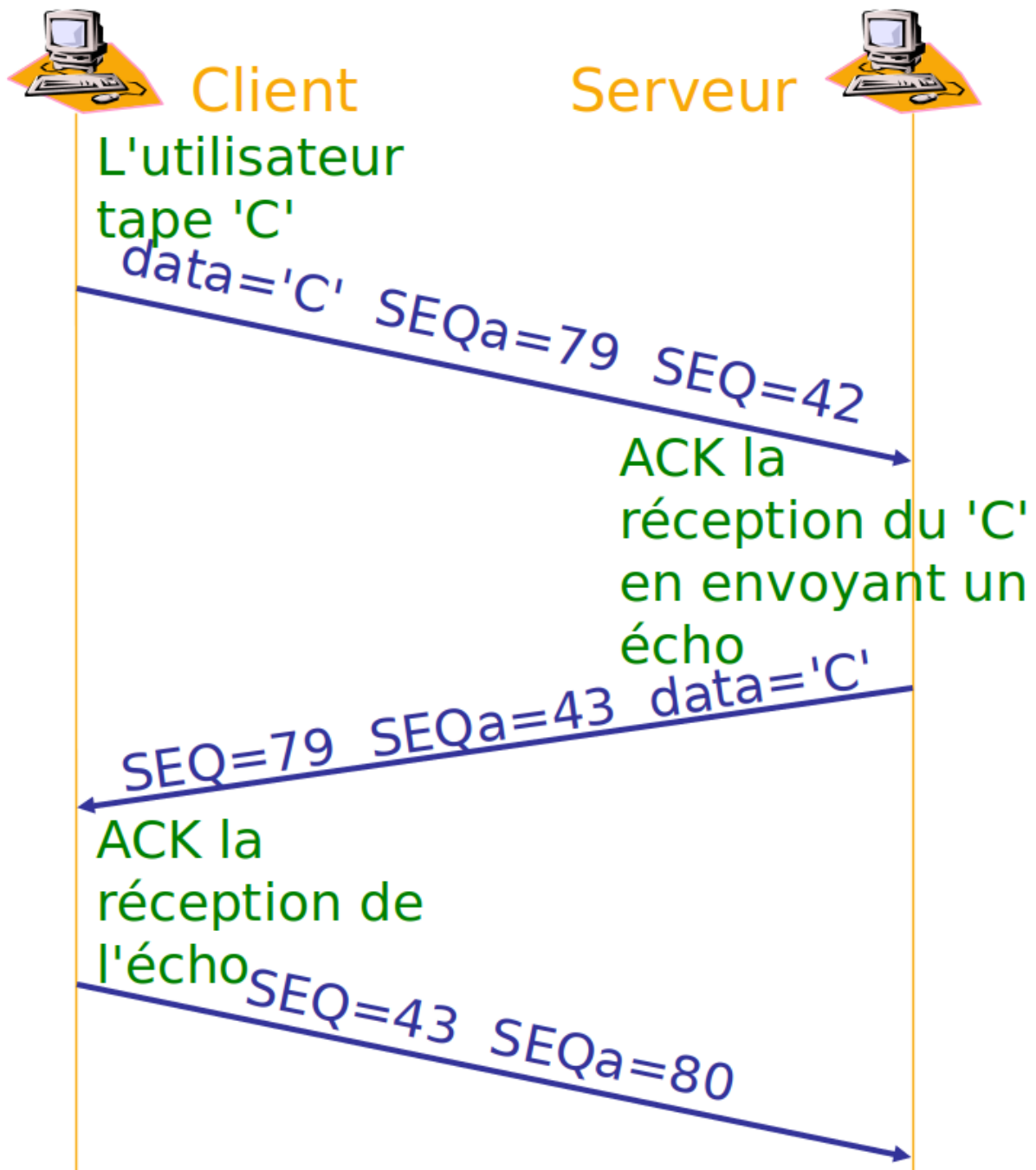
Le schéma ci-dessus illustre le principe des acquittements. Un client envoie la lettre 'C' au serveur. Le serveur acquitte tout de suite et en profite pour renvoyer l'écho du premier message : il renvoie la lettre 'C' qu'il a reçue. Ensuite, le client acquitte l'écho. L'application Telnet qui permet d'exécuter des commandes à distance fonctionne ainsi. Chaque lettre tapée au clavier sur le client dans la connexion Telnet est transmise au serveur caractère par caractère et le serveur renvoie un écho de la lettre tapée pour qu'elle s'affiche dans le terminal du client.

Les numéros de séquence SEQ et SEQa sur le schéma permettent de compter les octets envoyés (SEQ) et les octets reçus (SEQa). Ils sont transportés dans les champs NS et NR dans l'entête TCP. Lors de l'ouverture de la connexion, le numéro de séquence initial appelé **ISN** (Initial Sequence Number) est tiré aléatoirement. Il permet d'initialiser la valeur de SEQ et SEQa. Ensuite, à chaque fois qu'on envoie  $k$  octets, le numéro de séquence SEQ est incrémenté de  $k$ . De la même manière, à chaque fois qu'on reçoit  $k$  nouveaux octets, le numéro de séquence d'acquittement SEQa est incrémenté de  $k$ . SEQ et SEQa comptabilisent sur le client d'un côté et sur le serveur de l'autre les octets envoyés et reçus pour mettre en oeuvre la fiabilité.

### 1.2.9 Le contrôle de flux

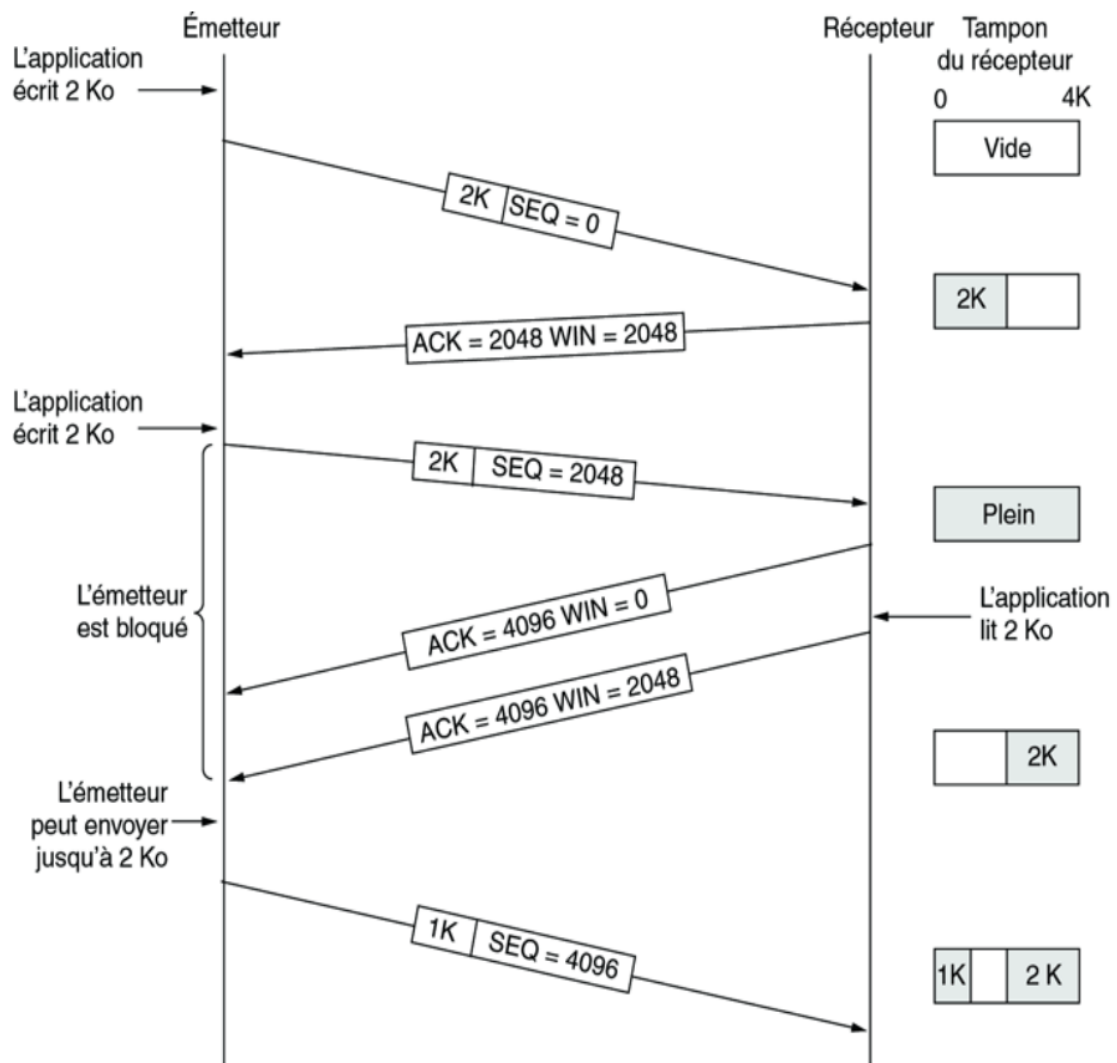
La figure ci-dessus illustre la mise en oeuvre du contrôle de flux. Dans l'entête TCP, le champ **WIN** permet au récepteur de dire combien d'octets il peut recevoir. Dans cet exemple, le tampon de réception fait 4 Ko. L'émetteur envoie 2 Ko dans un premier message. Ils sont acquittés par le récepteur qui indique alors qu'il a bien reçu 2048 octets et qu'il peut encore recevoir 2048 octets. Les données reçues sont conservées dans le tampon de réception jusqu'à ce que l'application vienne les lire.

TCP va adapter son débit et n'envoyer que 2048 octets dans son deuxième envoi puisque le récepteur ne peut pas en recevoir plus. Suite à cela, le récepteur acquitte ce deuxième envoi et indique à l'émetteur que



## Applications type telnet ou rlogin

Figure 8: Principe des acquittements



© Pearson Education France

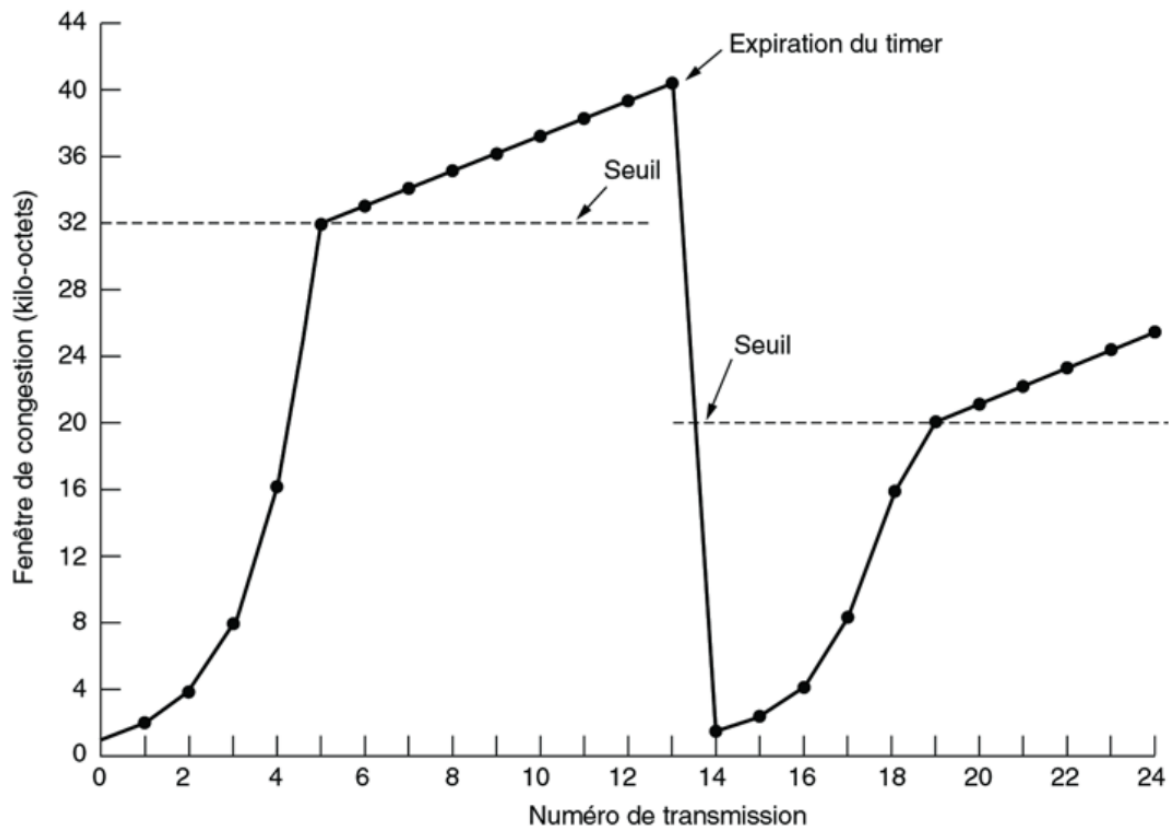
Figure 9: Mise en œuvre du contrôle de flux

son tampon de réception est plein (WIN=0). L'émetteur est bloqué, il ne peut plus envoyer jusqu'à ce que l'application côté récepteur vienne lire 2 Ko et libérer le tampon de réception. Cela déclenche l'envoi d'un nouvel acquittement vers l'émetteur indiquant qu'il peut à nouveau envoyer 2 Ko. Il envoie alors un troisième message contenant 1 Ko.

Le contrôle de flux est donc un facteur limitant le débit d'émission puisque TCP régule ses envois en fonction des capacités de réception du récepteur. Il permet d'éviter de perdre des données en réception.

### 1.2.10 Le contrôle de congestion

Le contrôle de congestion est un autre facteur limitant le débit d'émission. Il permet d'éviter de perdre des données en cas d'encombrement du réseau. L'algorithme de contrôle de congestion se déclenche dès lors que le *timeout* de retransmission arrive à zéro sur une émission c'est-à-dire dès lors qu'un message envoyé n'a pas été acquitté à temps. Dans ce cas, TCP considère qu'il y a une perte et donc qu'il faut ralentir le débit d'émission.



© Pearson Education France

Figure 10: Contrôle de congestion

L'algorithme est illustré par la figure ci-dessus. Au début, à l'ouverture de la connexion TCP, TCP envoie quelques octets, puis un peu plus, puis encore plus de manière exponentielle jusqu'à atteindre un certain seuil. Ensuite, il va continuer d'augmenter son débit et donc le nombre d'octets envoyés, mais cette fois de manière linéaire. Il continue d'augmenter son débit jusqu'à ce que le timeout de retransmission se déclenche pour l'envoi de ce message.

Dans ce cas-là, TCP réduit son seuil de congestion et reprend ses émissions avec un débit très faible. Puis il tente à nouveau d'augmenter son débit de manière exponentielle jusqu'au nouveau seuil puis de manière linéaire, jusqu'à atteindre à nouveau une éventuelle perte. Ainsi, TCP adapte son débit de manière dynamique en fonction de l'encombrement du réseau, c'est-à-dire de la congestion, mais aussi du contrôle de flux, c'est-à-dire des capacités de réception du récepteur.

Nous allons maintenant voir les protocoles de contrôle de l'Internet et en particulier les protocoles ICMP, ARP et DHCP ainsi que les utilitaires associés.

### 1.3 Le protocole ICMP

ICMP ([RFC 792](#)) est le protocole utilisé par les utilitaires `ping` et `traceroute` pour tester si le réseau Internet fonctionne depuis une source vers une destination. Ce protocole est assez simple. Il est basé sur l'envoi de requêtes qui vont être suivies d'une réponse.

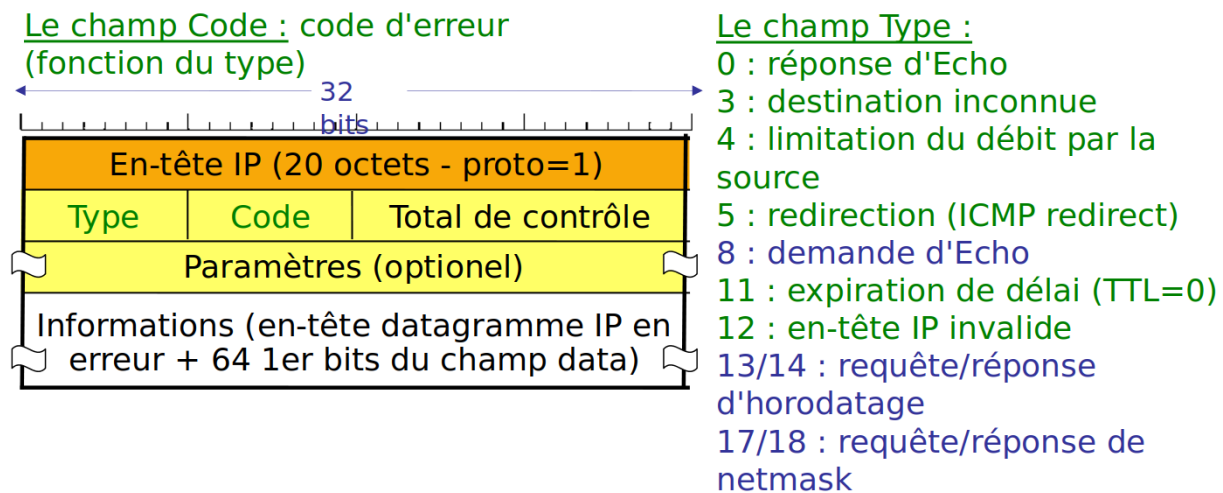


Figure 11: Entête ICMP

ICMP est encapsulé dans IP. Son entête fait huit octets et se compose des champs suivants :

- le champ **type** de la requête ou de la réponse. Les différentes valeurs prises par ce champ sont décrites à droite de la figure.
- Le champ **code** de la réponse. Le champ **total de contrôle**.

La section **data** qui généralement contient l'en-tête IP, suivie d'un certain nombre d'octets qui peuvent être soit positionnés par l'utilisateur, soit parce qu'ils se trouvent dans la mémoire à ce moment-là.

L'usage classique d'ICMP est pour faire du *ping*. Il s'agit d'une demande d'Echo (type=8) suivie de la réponse d'Echo (type=0). La demande est envoyée au destinataire qui répond. Le temps aller-retour est généralement mesuré. Le ping se fait depuis un terminal en ligne de commande, soit depuis un routeur, soit depuis un ordinateur. Un autre type de paquet ICMP est envoyé quand le TTL d'un paquet IP arrive à zéro sur un routeur. Dans ce cas-là, le routeur détruit le paquet et renvoie à l'émetteur un paquet ICMP avec le type=11 pour le prévenir du fait que le paquet a été perdu.

Il peut arriver également qu'une destination soit inconnue dans la table de routage d'un routeur. Dans ce cas, un paquet ICMP de type=3 (destination inconnue) est envoyé par le routeur pour prévenir l'émetteur.

Le *ping* permet de tester différents points de connexion dans le réseau et de dépanner quand le réseau ne fonctionne pas. Pinguer l'adresse locale sur localhost (127.0.0.1) permet de vérifier que l'architecture TCP/IP est en bon fonctionnement sur la machine locale. Pinguer sa propre adresse IP, c'est-à-dire l'adresse IP de

sa carte réseau permet de vérifier que la carte réseau est bien configurée et qu'elle répond bien aux requêtes qu'on lui envoie. Pinguer le prochain routeur et les suivants permet de sortir de son propre réseau et de vérifier que les routeurs qui nous interconnectent au reste d'Internet sont eux-mêmes bien configurés et que notre ordinateur est bien configuré pour les atteindre. Il est possible de pinguer une adresse IP destination qui est soit celle que l'on cherche à atteindre, soit des adresses IP intermédiaires que l'on peut contacter pour vérifier la connectivité.

Un autre utilitaire très utilisé est **traceroute**. Il permet également de tester une destination avec des requêtes et réponses ICMP tout en affichant chaque routeur intermédiaire qui est traversé sur le chemin.

Sur l'exemple ci-dessous, il y a deux routeurs intermédiaires pour atteindre la destination.

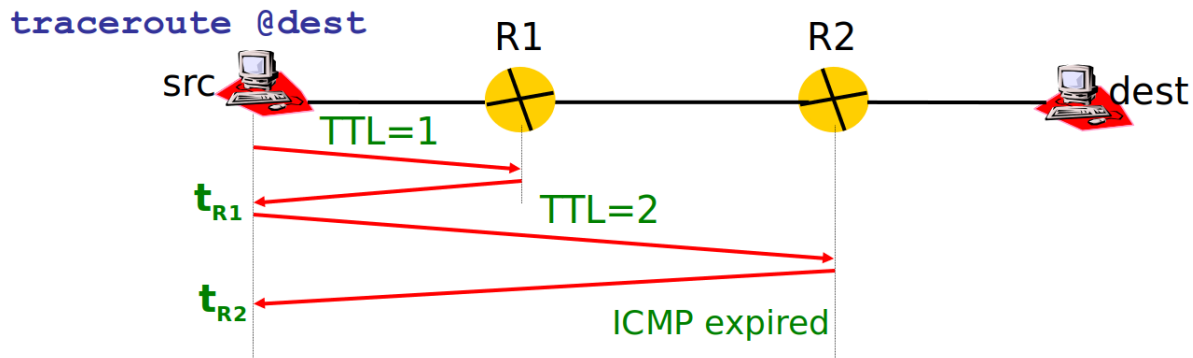


Figure 12: traceroute

Pour trouver les routeurs intermédiaires, le principe est le suivant. Un premier paquet ICMP est envoyé avec un TTL=1. Dès qu'il arrive sur le premier routeur, le TTL passe à zéro. Le routeur renvoie alors une réponse ICMP de type=11 (expiration du délai). Cela permet de trouver le premier routeur. Ensuite, traceroute envoie une nouvelle requête ICMP avec un TTL=2. Cela permet d'aller jusqu'au deuxième routeur qui va renvoyer un "Délai expiré" et ainsi de suite jusqu'à atteindre la destination finale. Cela permet donc de faire des allers-retours avec chaque routeur et de mesurer le temps qu'il faut pour l'atteindre et d'obtenir son adresse IP. *traceroute* affiche tous les routeurs traversés avec le temps qu'il faut pour les atteindre.

## 1.4 Le protocole ARP

Le protocole **ARP** (RFC 826) permet de faire la correspondance entre les adresses IP utilisées pour trouver le chemin sur Internet et les adresses MAC utilisées pour envoyer le signal d'une carte réseau à une autre carte réseau. C'est un protocole essentiel, car lorsque l'on veut se rendre quelque part sur Internet, on connaît l'adresse IP de destination, mais en réalité, il faut transférer des trames qui circulent sur toutes les liaisons intermédiaires. Ces trames sont transformées en un signal qui circule sur chaque liaison. Pour cela, on a besoin des adresses MAC pour aller d'une carte réseau à une autre carte réseau sur chacune des liaisons.

Comment fonctionne ARP ? Supposons qu'on ait la machine 1 avec l'adresse IP 192.31.65.7 qui veut pinguer la machine 2 avec l'adresse IP 192.31.65.5, toutes les deux sur le même LAN Ethernet. La machine 1 a besoin de connaître l'adresse MAC de la carte réseau de la machine 2, mais elle ne connaît que la sienne.

Pour récupérer l'adresse MAC de la machine 2, la machine 1 envoie un **broadcast** dans le réseau local en demandant "qui a l'adresse IP 192.31.65.5 ?". Il s'agit de la **requête ARP** qui est diffusée dans tout le réseau local pour demander à une machine distante, celle que l'on veut atteindre, de renvoyer son adresse MAC. La machine 2 renvoie alors son adresse MAC dans la **réponse ARP**.

Si la machine 2 n'est pas dans le même réseau que la machine 1, la table de routage de la machine 1 indique l'adresse IP du prochain routeur (prochain saut) pour atteindre la machine 2. Une requête ARP est alors envoyée pour trouver l'adresse MAC du prochain routeur qui va répondre en renvoyant son adresse MAC. Ce

routeur à son tour aura besoin de récupérer l'adresse MAC du routeur suivant grâce à une nouvelle requête ARP sur la liaison suivante et ainsi de suite jusqu'à atteindre la destination.

Pour éviter d'envoyer une requête ARP à chaque fois qu'on envoie un paquet IP, il y a un système de **cache ARP**. Le système conserve temporairement dans une table ARP les correspondances adresse IP / adresse MAC déjà connues. De plus, la correspondance adresse IP <-> adresse MAC de l'émetteur d'une requête ARP est incluse dans la requête pour que le récepteur, voire même toutes les machines du réseau local qui reçoivent le broadcast, mettent à jour leur cache ARP.

Pour être complet, voici le format des paquets ARP.

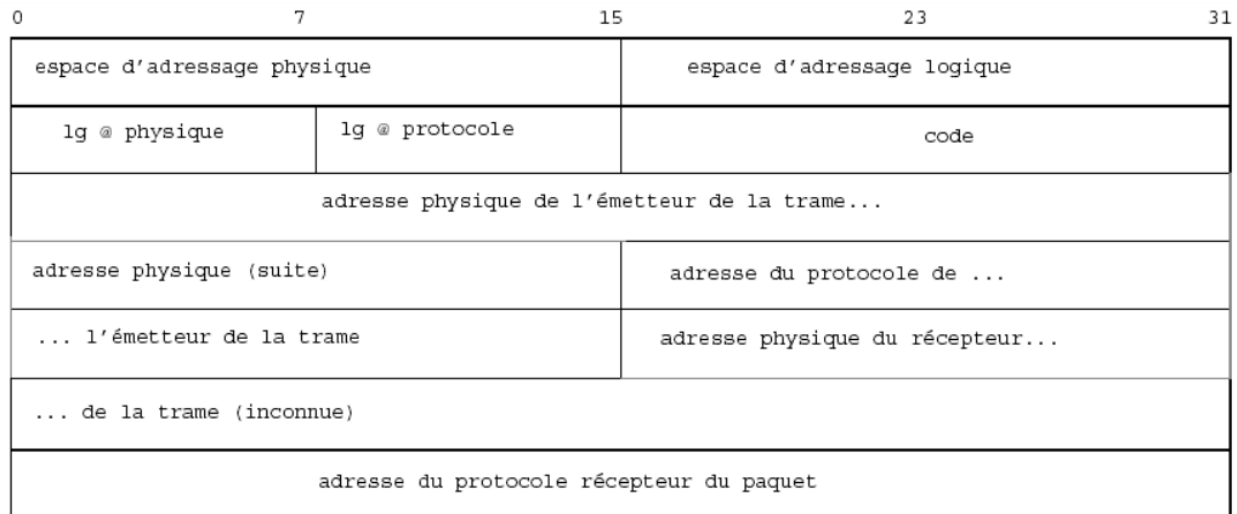


Figure 13: Format des paquets ARP

Un paquet ARP permet de stocker les adresses physiques (adresses MAC) et les adresses logiques (adresses IP) de la source et de la destination. Avant cela, le paquet ARP contient un entête de 8 octets qui indique l'espace d'adressage physique, généralement Ethernet, l'espace d'adressage logique, généralement IPv4 ou IPv6, puis la longueur des adresses, généralement 6 octets pour l'adresse physique et 4 octets pour l'adresse IPv4. Enfin, le champ **code** indique s'il s'agit d'une requête ou d'une réponse ARP, sachant que pour la requête, l'adresse MAC destination sera toujours l'adresse de diffusion **ff:ff:ff:ff:ff:ff**.

Un paquet ARP est directement encapsulé dans une trame Ethernet. Il fait 28 octets dans le cas d'une résolution d'adresse IPv4. La trame Ethernet contient alors 18 octets de bourrage pour atteindre la taille de trame minimale.

Dans tous les réseaux, les paquets ARP sont nombreux et circulent de manière très régulière.

## 1.5 Le protocole DHCP

Le protocole **DHCP** permet de distribuer dynamiquement des adresses IP à des ordinateurs dans un réseau local à partir d'un serveur DHCP. Par exemple, la box héberge un serveur DHCP. C'est donc elle qui configure dynamiquement l'accès à Internet de votre ordinateur.

Le fonctionnement du protocole DHCP est assez simple. Quand une machine démarre, au moment de l'activation de la carte réseau, si elle est configurée en DHCP, elle diffuse dans tout le réseau local une requête DHCP pour récupérer sa configuration réseau. Le serveur DHCP dispose d'une plage d'adresses IP disponibles et lui répond en lui indiquant une adresse IP, un masque réseau, et éventuellement l'adresse IP du prochain saut qui permet de sortir du réseau local ainsi que l'adresse du serveur DNS local qui prend en charge les requêtes DNS.

## 1.6 Fichiers de configuration et commandes UNIX

Pour terminer cette section, nous présentons quelques fichiers de configuration et commandes Unix qui interviennent dans la configuration réseau des ordinateurs.

- Le fichier `/etc/hosts` contient des associations nom local <-> adresse IP. Il permet de faire la résolution d'un nom en local avant de solliciter le DNS ou un annuaire LDAP. Dans ce fichier local, la correspondance nom <-> adresse IP n'a rien d'officiel donc l'utilisateur de la machine locale peut donner le nom qu'il veut à une adresse IP, à condition de posséder les droits `root` sur la machine.

```
127.0.0.1    localhost
```

```
# The following lines are desirable for IPv6 capable hosts
```

```
::1          ip6-localhost ip6-loopback
```

```
fe00::0      ip6-localnet
```

```
ff00::0      ip6-mcastprefix
```

```
ff02::1      ip6-allnodes
```

```
ff02::2      ip6-allrouters
```

```
#Following lines are auto-generated by script CREMI/ CFEngine3
```

```
10.0.230.9 mcgonagall.emi.u-bordeaux.fr mcgonagall
```

```
2001:660:6101:800:230::9 mcgonagall.emi.u-bordeaux.fr mcgonagall
```

```
...
```

- Le fichier `/etc/resolv.conf` permet de renseigner avec `nameserver` le ou les serveurs DNS qui seront sollicités pour faire la résolution de noms. Il permet aussi d'indiquer avec `search` les suffixes de recherche DNS c'est-à-dire les noms de domaines utilisés pour compléter un nom court comme `ssh` ou `www`.

```
nameserver 127.0.0.53
```

```
search emi.u-bordeaux.fr cremi.emi.u-bordeaux1.fr
```

Le fichier `/etc/protocols` contient les numéros des différents protocoles, comme par exemple 6 pour TCP, 1 pour ICMP, utilisés dans les entêtes :

```
ip      0    IP          # internet protocol, pseudo protocol number
```

```
hopopt  0    HOPOPT       # IPv6 Hop-by-Hop Option [RFC1883]
```

```
icmp    1    ICMP        # internet control message protocol
```

```
igmp    2    IGMP       # Internet Group Management
```

```
ggp     3    GGP        # gateway-gateway protocol
```

```
ipencap 4    IP-ENCAP   # IP encapsulated in IP (officially ``IP'')
```

```
st      5    ST         # ST datagram mode
```

```
tcp     6    TCP        # transmission control protocol
```

```
egp     8    EGP        # exterior gateway protocol
```

```
...
```

De la même manière, le fichier `/etc/services` contient les numéros de port des différents services, c'est-à-dire les applications connues sur Internet, comme par exemple 22 pour le serveur SSH, 21 pour le serveur FTP, etc. :

```
...
```

```
echo    7/tcp
```

```
echo    7/udp
```

```
discard 9/tcp      sink null
```

```
discard 9/udp      sink null
```

```
systat  11/tcp     users
```

```
daytime 13/tcp
```

```
daytime 13/udp
```

```

netstat      15/tcp
gotd         17/tcp      quote
chargen     19/tcp      ttytst source
chargen     19/udp      ttytst source
ftp-data    20/tcp
ftp         21/tcp
fsp         21/udp      fspd
ssh         22/tcp      # SSH Remote Login Protocol
telnet      23/tcp
smtp        25/tcp      mail
time        37/tcp      timserver
...

```

- Enfin, le fichier `/etc/inetd.conf` contient la liste des serveurs qui seront lancés, soit au démarrage de la machine, soit après lorsqu'ils seront sollicités par l'arrivée d'une requête, avec l'emplacement des exécutables associés à chaque service.

```

#
# <service_name> <sock_type> <proto> <flags> <user> <server_path> <args>
#
ftp;stream tcp nowait root /usr/sbin/ftpd ftpd
ntalk dgram udp wait root /usr/sbin/talkd talkd
telnet stream tcp6 nowait|DEBUG root /usr/sbin/telnetd telnetd -a
time stream tcp nowait root internal
time dgram udp wait root internal

```

Voici maintenant quelques commandes Unix liées à la configuration réseau avec des exemples d'utilisation :

```

toto@mcgonagall:~$ ping www.google.fr
PING www.google.fr(mrs08s20-in-x03.1e100.net (2a00:1450:4006:80f::2003)) 56 data bytes
64 bytes from mrs08s20-in-x03.1e100.net (2a00:1450:4006:80f::2003): icmp_seq=1 ttl=112 time=27.7 ms
64 bytes from mrs08s20-in-x03.1e100.net (2a00:1450:4006:80f::2003): icmp_seq=2 ttl=112 time=16.8 ms
64 bytes from mrs08s20-in-x03.1e100.net (2a00:1450:4006:80f::2003): icmp_seq=3 ttl=112 time=16.7 ms

```

```

--- www.google.fr ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 5ms
rtt min/avg/max/mdev = 16.685/20.376/27.695/5.177 ms

```

et le traceroute :

```

toto@mcgonagall:~$ traceroute www.labri.fr
traceroute to www.labri.fr (147.210.8.54), 30 hops max, 60 byte packets
 1 _gateway (10.0.230.254) 0.323 ms 0.301 ms 0.287 ms
 2 opnsense1.emi.u-bordeaux.fr (10.0.253.247) 0.133 ms 0.153 ms 0.176 ms
 3 b3a1.emi.u-bordeaux.fr (147.210.12.254) 0.725 ms 0.741 ms 0.766 ms
 4 www4.labri.fr (147.210.8.54) 0.761 ms 0.761 ms 0.777 ms
 5 www4.labri.fr (147.210.8.54) 0.729 ms 0.730 ms *

```

La commande `arp` permet de visualiser la table ARP, c'est-à-dire les correspondances qui ont déjà été trouvées entre les adresses IP et les adresses MAC. C'est donc un cache qui permet de ne pas refaire les demandes de manière systématique quand elles se suivent.

```

nbonicho@mcgonagall:~$ /sbin/arp

```

| Adresse                      | TypeMap | AdresseMat        | Indicateurs | Iface |
|------------------------------|---------|-------------------|-------------|-------|
| quirrell.emi.u-bordeaux.fr   | ether   | bc:97:e1:a5:99:42 | C           | bond0 |
| infini1.emi.u-bordeaux.fr    | ether   | 00:26:b9:75:c5:ea | C           | bond0 |
| trellawney.emi.u-bordeaux.fr | ether   | f0:4d:a2:3f:15:05 | C           | bond0 |
| hagrid.emi.u-bordeaux.fr     | ether   | b0:26:28:c1:6e:fa | C           | bond0 |

|                           |       |                   |   |       |
|---------------------------|-------|-------------------|---|-------|
| _gateway                  | ether | 58:20:b1:b1:23:00 | C | bond0 |
| ombrage.emi.u-bordeaux.fr | ether | bc:97:e1:a5:f5:28 | C | bond0 |
| buster.emi.u-bordeaux.fr  | ether | 00:50:56:af:6b:89 | C | bond0 |

La commande `host` permet de faire des requêtes DNS en ligne de commande :

```
toto@mcgonagall:~$ host www.google.fr
www.google.fr has address 142.250.201.35
www.google.fr has IPv6 address 2a00:1450:4006:80f::2003
```

La commande `netstat` permet d'obtenir des statistiques sur le nombre de paquets envoyés, le nombre de paquets reçus, les sockets ouvertes, les erreurs ou les collisions qui ont pu se produire au niveau de la carte réseau.

```
nbonicho@mcgonagall:~$ netstat -i
```

| Iface | MTU   | RX-OK    | RX-ERR | RX-DRP | RX-OVR | TX-OK   | TX-ERR | TX-DRP | TX-OVR | Flg   |
|-------|-------|----------|--------|--------|--------|---------|--------|--------|--------|-------|
| bond0 | 9000  | 11567989 | 0      | 33     | 0      | 9541898 | 0      | 0      | 0      | BMmRU |
| eth0  | 9000  | 5840108  | 0      | 0      | 0      | 25218   | 0      | 0      | 0      | BMsRU |
| eth1  | 9000  | 5727881  | 0      | 0      | 0      | 9516680 | 0      | 0      | 0      | BMsRU |
| lo    | 65536 | 256514   | 0      | 0      | 0      | 256514  | 0      | 0      | 0      | LRU   |

Cela permet également d'afficher la table de routage.

```
nbonicho@mcgonagall:~$ netstat -r
```

Table de routage IP du noyau

| Destination | Passerelle | Genmask       | Indic | MSS | Fenêtre | irtt | Iface |
|-------------|------------|---------------|-------|-----|---------|------|-------|
| default     | _gateway   | 0.0.0.0       | UG    | 0   | 0       | 0    | bond0 |
| 10.0.230.0  | 0.0.0.0    | 255.255.255.0 | U     | 0   | 0       | 0    | bond0 |

C'est vraiment la commande qui est très utilisée par les administrateurs système et réseaux.

Enfin, pour rappel, les commandes `ip` permettent de configurer les cartes réseaux et la table de routage. Par exemple :

```
nbonicho@mcgonagall:~$ ip addr add 192.168.9.3/22 dev eth0
nbonicho@mcgonagall:~$ ip link set up dev eth0
nbonicho@mcgonagall:~$ ip route add 0.0.0.0/0 via 192.168.8.1
nbonicho@mcgonagall:~$ ip route show
```

| Destination | Masque        | Passerelle  | Interface |
|-------------|---------------|-------------|-----------|
| 192.168.8.0 | 255.255.252.0 | -           | eth0      |
| 0.0.0.0     | 0.0.0.0       | 192.168.8.1 | eth0      |